

Track 4: Java Advanced Concepts

Exercise 14: Principles of Software Logging Instruction: This is a hands-on seatwork. Follow the instructions given below.	Hours: 2 hours Create a Maven Project <FamilyName>Ex14 and create logs inside the project to contain all your log files.
--	--

- A. Maven.** Create a Maven project. Build your pom.xml. Include, together with your **JUnit 5**, the latest **Log4J2 libraries** in your dependencies.
- B. Getting to Know Logging.** Examine the simple code below. Add the necessary logging and replace all **System.out** lines with Log4J2 logs using **FileAppender**.

```
public class SimpleApp {

    public int returnOne() throws Exception{
        try {
            System.out.println( "Here's One!" );
            return 1;

        } catch (Exception ex) {
            throw new Exception("Napakadali nito!");
        }
        return 0;
    }
}

public class TestSimpleApp {

    @Test
    public void testApp() {
        SimpleApp app = new SimpleApp();
        assertTrue("app didn't return 1", app.returnOne() == 1);
    }
}
```

Logs must have the format below:

INFO - 2008-09-06 10:51:44,817 - SimpleApp.java - Here's One!

and must be logged in **simple_app.log**. Apply the necessary log levels.

- C. Re-code your TrapArray** and add some logging mechanism which does the following:
- a. logs when the loop starts like this:

INFO: TrapArray starts. (2008-09-06)

- b. logs when array of integers is successfully created in this manner:

INFO: Array of 3 integers created. (2008-09-06)

c. logs every exception such as:

```
ERROR: Division by zero. (2008-09-06)
```

Use Log4J2's rolling FileAppender **triggered by time per second** to log all these messages to a log file named **traparray.log**. Apply the necessary log levels.

- D. Re-code your **ShoppingCart** reflecting all your mock proposals to the original version. Afterwards, add a Log4J2 logger that will log the following:
- a. every start of the execution of getBalance(), **addItem()**, **removeItem()**, **getItemCount()** and **empty()**
 - b. every instantiation
 - c. every end of the execution of all the three methods
 - d. log the return value before it reaches the **return** statement

Create the necessary message for the logs but following this format:

```
2016/06/20 19:24:45,076 [ ERROR ] ChekcStr - Invalid value !!
```

Use Log4J2's rolling FileAppender **triggered by file size up to 10kb with 10 backups** and store all logs in **chekcstr.log** file. Apply the necessary log levels.

- E. Consider the classes below and study its process flow. Re-code these classes to include Log4J2 logging during their executions. Create a test class **TestVendingMachine** to execute the classes. After running and studying its output, create a Log4J2 logger that will do the following:
- a. replace all the **System.out** and **System.err** with Log4J2 message
 - b. log every start and end of method execution
 - c. log the type of coin for every coin instance

Use **HTMLLayout** to log all the messages above in an html file **vendo_log.html** and must look like this in an HRMS:

Thread	Level	Logger	Message
main	ERROR	htmlLogger	Employee Information Update Failed
main	INFO	htmlLogger	New Account Created

Do not forget to apply the necessary Log Levels.

```
public class Coin {
    public static final int PENNY = 1;
    public static final int NICKEL = 5;
    public static final int DIME = 10;
    public static final int QUARTER = 25;
    public static final int HALFDOLLAR = 50;
    public static final int SILVERDOLLAR = 100;

    public static final int INVALID = 0;

    private int value;
```

```

public Coin( String coin ){
    String toUpperCoin = coin.toUpperCase();
    if ( toUpperCoin.equals( "PENNY" ) ) value = PENNY;
    else if ( toUpperCoin.equals( "NICKEL" ) ) value = NICKEL;
    else if ( toUpperCoin.equals( "DIME" ) ) value = DIME;
    else if ( toUpperCoin.equals( "QUARTER" ) ) value = QUARTER;
    else if ( toUpperCoin.equals( "HALFDOLLAR" ) ) value = HALFDOLLAR;
    else if ( toUpperCoin.equals( "SILVERDOLLAR" ) ) value = SILVERDOLLAR;
    else value = INVALID;
}

public int getValue() {return value;}

public String toString() { return new String(Integer.toString(value)); }
}

//VendingMachine
public class VendingMachine {
    private int totValue;
    private int currValue;
    private boolean enabled;
    private static final int COST = 75;

    public VendingMachine() {
        totValue = 0;
        currValue = 0;
        enabled = false;
    }

    public void insert( Coin coin ) {
        int value = coin.getValue();
        if ( value == 0 ) {
            System.err.println( "illegal value");
        }
        else {
            currValue += value;
            System.out.print( "Current value = " + currValue );
            if (currValue >= COST) {
                enabled = true;
                System.out.println( " (sufficient credit)" );
            }
            else System.out.println( "" );
        }
    }

    public void returnCoins() {
        if ( currValue == 0 ) {
            System.err.println( "no coins to return" );
        }
        else {
            System.out.println( "Take your coins" );
            currValue = 0;
            enabled = false;
        }
    }

    public void vend() throws Exception {
        if (enabled) {
            System.out.println( "Get your drink");

```

```
        totValue += COST;
        currValue -= COST;
        if (currValue == 0) enabled = false;
        System.out.println( "Current value = " + currValue );
    }
    else {
        System.out.println( "Not enough credit: add " + (COST - currValue));
    }
    if (currValue < 0) throw new Exception("Error: negative credit!");
}
}
```